



Distributed Systems and Applications (DSA612S)

Assignment 1: Distributed Library and Resource Management System

Release Date: 20 July 2026 (Week 1)

Due Date: 14 September 2026 at 23:59 (Week 9)

Total Marks: 100

Submission Format: GitHub/GitLab Repository Link on eLearning

Question 1 : RESTful APIs — Library and Resource Management System (50 Marks)

1. Project Overview

The Ministry of Higher Education, Training and Innovations requires a comprehensive, distributed **Library Management System** shared across multiple campuses of all registered institutions of higher learning. This system must track books, electronic resources (laptops, thin clients, etc.), and physical spaces (labs and meeting rooms). Each resource includes important details such as its **tag, name, description, institution, site/campus, date acquired**, and **status** (AVAILABLE, LOANED_OUT/OCCUPIED, UNDER_MAINTENANCE, or DISPOSED). You are tasked with designing and implementing a **RESTful API backend** using the **Ballerina** programming language and a **client-side interface** using a Ballerina client (command-line based) to interact with the API.

2. Technical Specifications

- **Backend:** Ballerina RESTful Service.
- **Frontend:** Ballerina client that correctly interacts with the service.
- **Database:** using **Map** or a **table**
- **Communication:** Inter-process communication via **HTTP/REST**.
- **Unique Identifier:** All assets must be identified by a unique **assetTag**.

3. Functional Requirements

A. Backend API Service (Ballerina) Your API must follow **RESTful principles** and support the following operations:

- **Asset Management:** CRUD operations (Create, Read, Update, Delete) for library resources.
- **Institution Filtering:** Retrieve a list of assets belonging specifically to one institution.
- **Maintenance & Overdue Checks:** Identify assets where the maintenance due date has passed.
- **Component & Schedule Management:** Add/remove components for complex assets (e.g., a printer motor) and manage booking/servicing schedules.
- **Work Orders & Task Tracking:** Open, update, or close work orders for faulty resources, including managing sub-tasks (e.g., "replace screen").

B. Ballerina client service. The client must interact with the Ballerina API to demonstrate the following:

1. **Loaning & Booking:** A user interface to loan an asset or book a meeting room/lab.
2. **Global View:** Display a comprehensive list of all assets across the ministry.
3. **Campus View:** Filter resources by specific institution or site/campus.
4. **Overdue Dashboard:** A view for staff to quickly identify overdue items.
5. **Schedule Manager:** Interface to add or modify servicing schedules.

4. Example of an Updated Asset Payload

```
{
  "assetTag": "NUST-LIB-3DP-001",
  "name": "Pro-Series 3D Printer",
  "description": "High-precision laboratory printer for simulation and
  prototype development.",
  "institution": "Namibia University of Science and Technology",
  "site": "Main Campus - Innovation Lab",
  "status": "AVAILABLE",
  "dateAcquired": "2024-03-10",
  "components": [
    {
      "compId": "C101",
      "name": "High-Torque Stepper Motor",
      "description": "Main motor for X-axis movement."
    }
  ],
  "schedules": [
    {
      "scheduleId": "SCH-882",
      "type": "MAINTENANCE",
      "dueDate": "2026-09-01",
      "description": "Quarterly calibration and nozzle cleaning."
    }
  ],
  "workOrders": [
    {
      "orderId": "WO-554",
      "status": "OPEN",
      "description": "Nozzle heat-bed failure",
      "tasks": [
        {"taskId": "T1", "description": "Check thermal sensor connectivity."}
      ]
    }
  ]
}
```

}

To implement the data models effectively, refer to the above sample payload in JSON format and use the **Ballerina by Example** documentation

5. Mark Allocation

Deliverable	Description	Marks
Working Solution	Correct setup, compilation, and repository structure.	10
API Implementation	Full implementation of CRUD, filtering, maintenance checks, and work orders. • Create and manage resources - add, update, look up, or remove (5 marks) • View all assets - retrieve the full list (2 marks) • View assets by institution and site - filter by category (3 marks) • Check for item status and booking schedules (5 marks) • Manage institutions- add/remove institutions from listings (5 marks) • Manage schedules -add/remove schedules for a resources (3 marks) • Error and Wrong API calls handling (2 marks)	25
Database Integration	Effective use of Map/Table with assetTag as the unique key.	5
Client Implementation	Functional Ballerina client that demonstrating loaning, viewing, and scheduling. BONUS MARKS FOR IMPLEMENTATION USING WEB/MOBILE INTERFACE(~10 MARKS)	10

Question 2: Remote Invocation(gRPC): RENTAL ACCOMMODATION SYSTEM (50 Marks)

Objective: To design and implement a distributed system for managing property listings and bookings using the **gRPC** protocol and the **Ballerina** programming language.

1. Project Overview

The Ministry of Tourism requires a distributed platform to manage short-term accommodation rentals across various regions. The system supports two primary user roles—**Host** and **Guest**. Hosts manage their property listings, while Guests can browse, search, and book accommodations for specific dates.

2. System Operations

You must define a **Protocol Buffer (.proto)** contract that supports the following operations:

- **add_property (Simple RPC):** A Host registers a new accommodation listing (property name, location, property type, price per night, and status). The system returns a unique `property_id`.
- **create_users (Client-side Streaming):** Multiple user profiles (Hosts or Guests) are created and streamed from the client to the server. The server sends a single confirmation once all users are registered.
- **update_property (Simple RPC):** A Host updates listing details (e.g., price or status) using the `property_id` as the unique key.
- **remove_property (Simple RPC):** A Host deletes a listing. The server responds with the **new full list** of available properties in that Host's region.
- **list_available_properties (Server-side Streaming):** A Guest requests properties available for rent. The server **streams** back property details one by one, optionally filtered by location or price range.
- **search_property (Simple RPC):** A Guest looks up a specific property by its `property_id`. If available, the server returns full details; otherwise, it returns a "Not Available" status.
- **book_property (Simple RPC):** A Guest selects a property and provides check-in and check-out dates. The server performs basic validation (e.g., end date is after start date) and adds the request to a temporary "booking cart".
- **confirm_booking (Simple RPC):** Finalizes the booking. The server must:
 - Verify the property is still available for the requested dates (ensure no date overlaps).
 - Calculate the total cost (Price per night × Number of nights).
 - Return a booking confirmation and clear the Guest's temporary request.

Please be aware that you have the freedom to include additional fields in your records if you believe they would enhance the performance and overall quality of your system

3. Technical Requirements

- **Contract Definition:** Define the service and messages in a `.proto` file.
- **Server Implementation:** Implement the logic in **Ballerina**. The server must handle concurrent requests and maintain the state of properties and bookings (use Ballerina `maps` or `tables` for the in-memory data store).
- **Client Implementation:** Implement a **Ballerina gRPC client** that demonstrates all the above operations. The client should be able to handle streaming responses for listing properties.

4. Mark Allocation (50 Marks)

Deliverable	Description	Marks
Protocol Definition	Buffer Correct <code>.proto</code> file defining all services, messages, and streaming types.	15
gRPC Implementation	Client A functional Ballerina client capable of invoking all remote functions.	10
gRPC Implementation	Server Comprehensive server-side logic handling data persistence, validation, and price calculation.	25

Submission & Academic Integrity

- **Group Work:** This is a group assignment (4 - 8 members). All members must be contributors in the repository.
 - **Bonus Marks:** additional marks will be allocated for creative and innovative system designs.
 - **Plagiarism:** 100% AI-generated code will be awarded a zero. AI tools should only be used as a guide.
 - **Presentation:** Groups will be required to present and defend their solution to receive marks.
 - **Deadline:** Commits made after **14 September 2026, 23:59** will not be accepted.
-